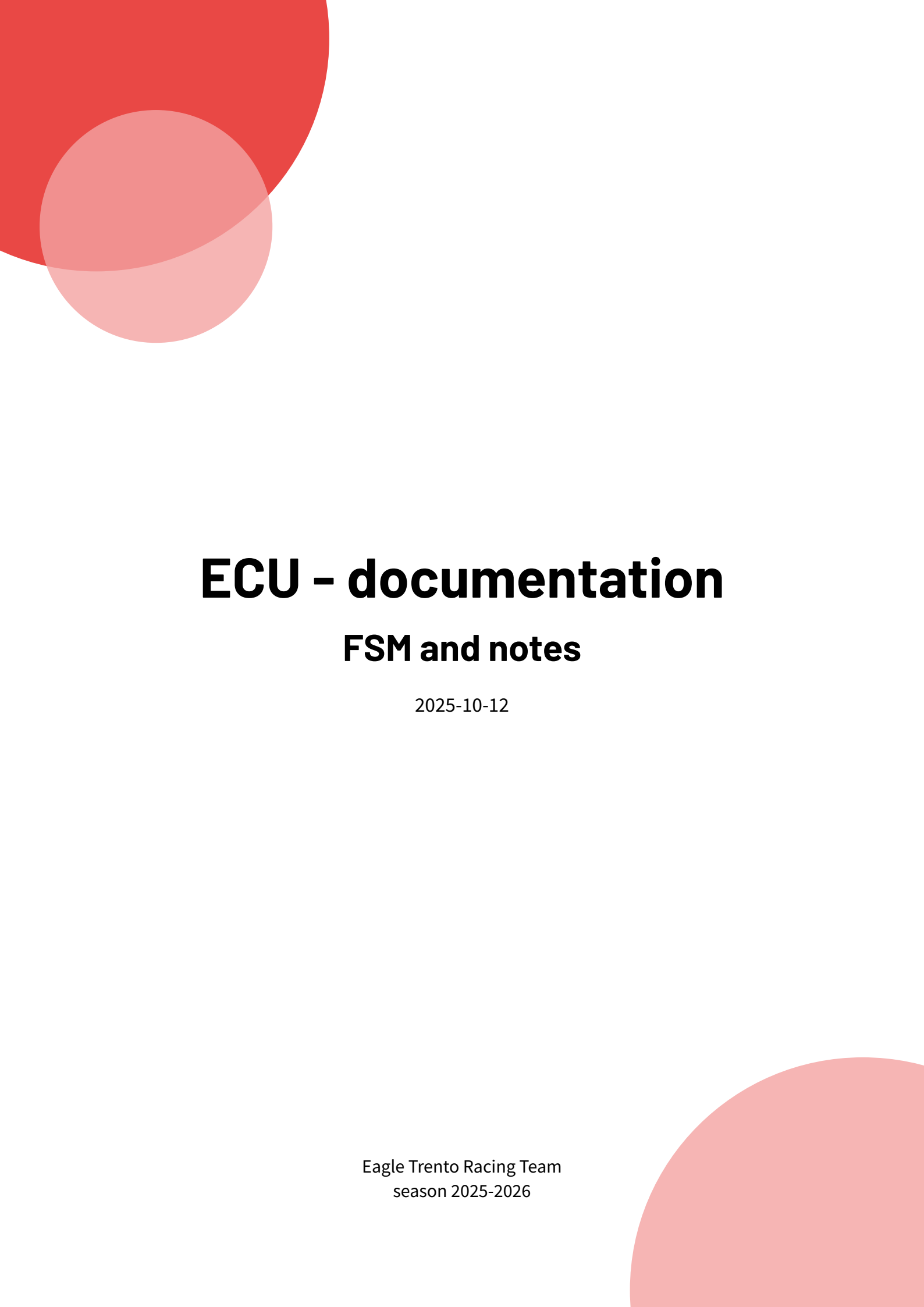




ECU - documentation

FSM and notes

2025-10-12

Eagle Trento Racing Team
season 2025-2026

I. Introduction

This document provides an overview of the design and ongoing implementation of the Finite State Machine (FSM) developed for the Electronic Control Unit (ECU), a key component in the new season's Formula Student vehicle.

As the project is still in progress, **the content of this document is subject to change**. Future updates may affect not only the written material but also the design decisions and implementation strategies described herein.

The intent is to maintain this document as a living reference that reflects the system's continuous evolution throughout the season.

II. Rulebook

As with every other subsystem in the car, the ECU must comply with the official Formula Student rules. Among the various competitions, the **Formula Student Germany (FSG) rulebook** is typically regarded as the **main reference** — effectively the “Bible” of the project ([link](#))

After analyzing the relevant sections, the following points have been identified as those **most closely related to the ECU's design and development**. It is important to note that not all of these have a direct software implication, but all influence the ECU's overall architecture or safety logic.

- T 1.4 Driving Mode Definitions

Manual mode: when the vehicle is driven by a human driver, the Autonomous System (AS) must be deactivated (ASMS off).

Autonomous mode: when the AS is active, no driver is allowed inside the vehicle.

- T 6.3 Brake Light

The brake has to be illuminated if and only if the brake is actuated, with a red light.

- T 14.2 Data Logger

A data logger will be provided and has to be installed in the car. The team needs to provide two sets of signals to the data logger.

- T 14.4 Shutdown Circuit

It can be closed by the AS only if all are fulfilled:

- manual driving and the AS has checked that ASB is deactivated
- autonomous driving, ASMS is switched on and sufficient brake pressure is built up

- T 14.8 Autonomous System Status Definitions

There is a flowchart to determine the status of the AS which can be found at [page 67](#).

- T 14.9 Autonomous System Status Indicators

Similar for the TSAL, based on the status of the AS a led has to be turned on. In particular, the emergency status has also to activate an intermitted sound (T 14.9.5)

- T 15.3 Functional Safety

An initial check has to be performed to ensure that ASB is able to build up brake pressure as expected, before AS transitions to “AS Ready”. The ASB and its SCS must be continuously monitored for failures and in case, trasnit to the safe state (brake pressed, standstill).

- EV 4.6 Data Logger

Similar for the AS, a data logger will be provided.

- EV 4.10 Tractive System Active Light

A led that indicate the TS status.

For more info about color, frequency, check the section.

- EV 4.12 Ready-To-Drive Sound

Generate a specific sound when the car is in r2d mode.

The **2026 rulebook is currently under development (last checked: 8 October 2025)**. However, no major changes are expected concerning ECU logic or FSM behavior.

III. Embedded meetings

Even though the ECU has its own project-specific characteristics, its core logic is built upon the FSM, which serves as a base module shared among all other embedded projects.

During the meetings held over the past month, several FSM states were discussed and categorized as either mandatory or optional, depending on their current and future relevance to system functionality.

For the **mandatory states**, we hav

- INIT
- IDLE
- FATAL

As for the **optional (additive) states**, the following may be included depending on future developments:

- FLASH: if the possibility to flash a microcontroller via CAN or other peripheral is introduced.
- PAUSE: if required by SPEC to change the boards parameters at runtime.

Aside from these guidelines, no strict implementation rules were provided, allowing flexibility in how the new FSM can be structured and optimized.

Nevertheless, the projects from past seasons remain valuable references, both as a foundation for designing new FSMs and as a resource for improving existing implementations.

IV. FSM Design

The FSM represents the core logic of the ECU — without a solid design, it would be impossible to develop reliable, maintainable, and high-quality code.

Among all the embedded systems, the ECU FSM is arguably the most challenging to design due to its central role and the large number of controls it must handle in real time.

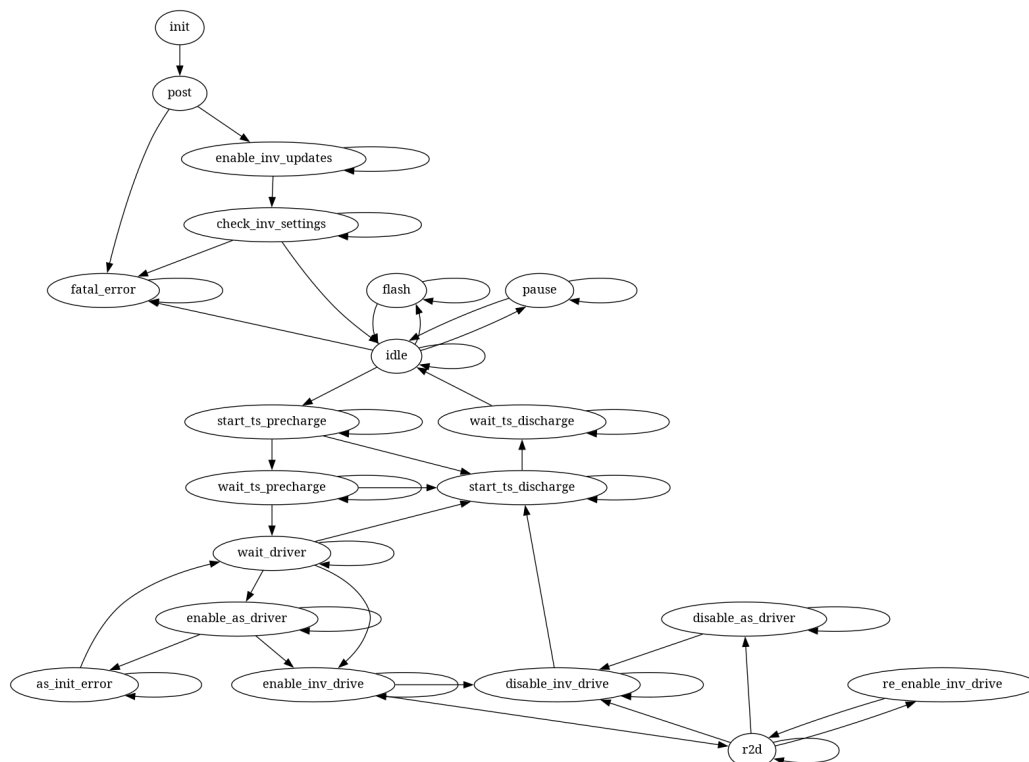
As mentioned in the previous section, the FSM developed in past seasons has been extremely valuable as a starting point. It provided a clear understanding of the ECU's expected behavior, the peripherals it must interface with, and the operational flow required to safely manage the vehicle.

Since the ECU must still perform the same fundamental tasks as in previous years, the natural choice was to reuse the existing FSM as a foundation and enhance it with new features and states. This approach offers two key advantages:

- It leverages a design that has already proven to work reliably, even if parts of the codebase are not fully readable or optimized;
- It significantly reduces development time, which is crucial given that a **working prototype is expected by March**.

A **major design consideration** for the 2025–2026 season is the **integration of the Autonomous System (AS)** into the ECU workflow. This introduces new requirements: the FSM must now handle both manual and autonomous driving modes, ensuring that safety and control logic remain consistent across both.

Figure 1 — “First version”



The image above illustrates the first version of the FSM, updated with the new states related to the Autonomous System (AS). Most of the states are inherited from previous seasons, with the addition of the optional ones (FLASH and PAUSE), which can be omitted if not required in the final implementation.

As mentioned earlier, **this version should not be considered final**, as the integration of the AS into the ECU workflow is still under definition.

Any future modifications or updates to the FSM design will be documented in this report as development progresses.

States description

The next section analyzes the objective and purpose of the most important states, to better understand how the overall execution flow evolves.

- **post (Power On Self Test):**
Similar to what a PC does when turned on, this state checks that all peripherals and communication channels are correctly enabled before continuing. Any issue detected here should be immediately reported, as it would compromise the correct execution of the ECU.
- **check_inv_settings:**
A critical state, since an incorrect configuration of the inverters will cause the ECU to transition into the `fatal_error` state with no possibility of recovery.
- **idle:**
Although it may seem like nothing happens here, this state ensures that the car has no output power, meaning the vehicle is stopped. The Tractive System (TS) will be precharged only when an explicit request is received by the ECU.
- **start_ts_precharge:**
Requests the BMS to activate the TS precharge phase. If an acknowledgment (ACK) is not received within a specified timeout, the system transitions to the discharge state, which in turn leads back to idle.
- **wait_ts_precharge:**
At this point, the BMS has acknowledged the precharge request, and the ECU waits for the process to complete. If it takes too long (timeout), the system aborts by requesting discharge.
- **wait_driver:**
This is where the main distinction between manual and autonomous operation occurs. To simplify the design, the following logic is adopted: if the driver is human, the behavior follows past seasons (waiting for brake pedal activation); if the driver is the AS, the ECU waits for specific conditions indicating readiness from the autonomous system.
- **enable_as_driver:**
When the AS is the driver, this state ensures that all necessary peripherals are initialized or activated before proceeding.

Why is this check not included in POST?

The main reason is that ensuring the human driver can operate the car takes priority. If the AS cannot initialize properly, the system transitions to `as_init_error` and then falls back to `wait_driver` without triggering a `fatal_error`. (Adding a debug layer here could be helpful.)

- **r2d:**
The Ready-to-Drive state, where the car becomes drivable, regardless of whether the driver is a human or the AS.
- **disable_as_driver:**
Slightly different from the procedure of the human driver, we might require to do some actions before requesting the disabling of the inverters and the discharge of ts.
- **re_enable_inv_drive:**
When the car is moving, it might happen that one (or even both) inverter gets deactivated, it is important to requesting its activation right after.

V. Peripherals and functions

Regardless of the final FSM design, several core functionalities and peripherals must be implemented in the new ECU to ensure compliance with vehicle requirements.

After analyzing the previous ECU codebase, past schematics, and the new hardware design, the following key (and NOT final) requirements were identified:

- **Buzzer control:** emits the Ready-to-Drive (R2D) sound.
The RTD pin generates the tone, while the AUX pin (connected to a timer) controls the timing and duration of the sound.
- **CAN interfaces:** required to transmit and receive CAN frames for communication with other boards and subsystems.
- **PTT (Push-To-Talk):** enables voice communication between the driver and the team.
- **Enable pin for Raspberry Pi:** allows the ECU to request a graceful shutdown of the Raspberry Pi in case of critical issues, preventing data corruption.
- **Pedal board interface:** verifies driver presence (wait_driver state) before allowing the system to transition to the R2D state.
- **BMS interface:** ensures that the Tractive System (TS) is enabled and monitors for any BMS related errors.
- **Brake and lighting peripherals:** handle the Tractive System Active Light (TSAL) and related brake control signals.
- **Debugging interface (UART):** provides runtime diagnostic output for monitoring and debugging.
- **Timer callbacks:** support event-driven tasks and time-based operations within the FSM.